

Agentic AI for Economic Research

Concepts, tools, workflows and responsible use

Marco Le Moglie

Catholic University of the Sacred Heart

Agentic Economic Research seminar | 1 July 2026

Mini-course for PhD students and faculty | 4 hours

Slides source on GitHub

Who I am, and what this course is *not*

Who I am

- Applied economist, Catholic University of the Sacred Heart.
- Daily user of agentic AI inside a research workflow: papers, replication, grants, teaching.
- Not a CS researcher or an AI safety specialist.

Why this course exists

- Doctoral students and faculty keep asking the same questions about tools, costs, privacy, autonomy.
- Most online material is either advertising or alarmism.

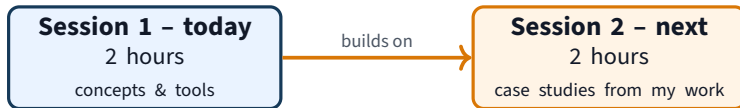
This course is **not**

- A coding tutorial.
- A benchmark comparison of language models.
- A discussion of artificial general intelligence.
- A claim that agents will replace researchers.

It is

- A practical map for using agents in economics research without losing scientific control.

Course roadmap: four hours, two sessions



1. What is an agent? Vocabulary.
2. Four families of interfaces.
3. Cost, privacy, governance.
4. Reliability and control.
5. Architecture: rules, memory, MCP, sub-agents.
6. A teaser of the case studies.

- A. Literature review.
- B. Data and codebook documentation.
- C. Stata / R / Python debugging.
- D. Tables, robustness, specification curves.
- E. Referee reports, grants, teaching prep.

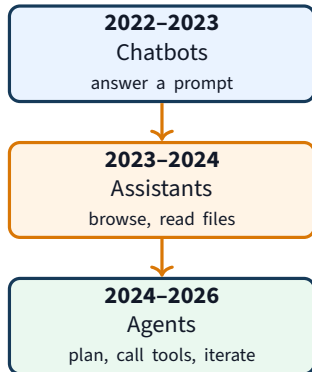
A practical map, not a technical manual.

Part 1 – Why now

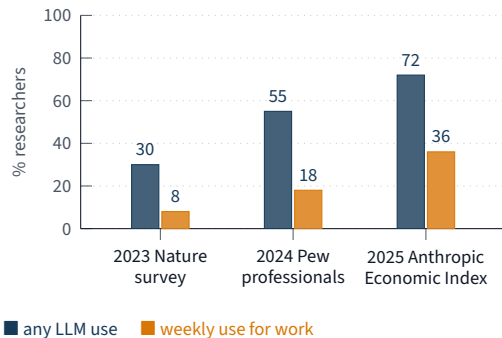
From answering to acting

- AI is moving from **answering questions** to **taking actions**.
- Research work is full of repeatable tasks: searching, coding, checking, documenting.
- The hard question is no longer “can it help?” but **where should we trust it?**

Agents change what we delegate.



Adoption among researchers is no longer marginal



- Roughly **1 in 3** researchers used an LLM at least once already in late 2023.
- By 2025, surveys converge on a **majority** using LLMs at work and roughly **1 in 3** weekly.
- The frontier is shifting from *chat use* to *agentic use* (tool calls, code, web).

Source: Van Noorden & Perkel, Nature 2023; Pew Research 2024 survey of US professionals; Anthropic Economic Index (2024a) 2024–2025 update.

Three things that changed since “ChatGPT 2023”

1. Tools

browser, files,
code, shell, search
the model can *do* things

2. Memory

session, project,
persistency
the model carries state

3. Autonomy

plan, loop, retry
the model decides next steps

2023 was about *generation*. 2025–2026 is about *delegation*.

Why economists in particular should care

Our work is unusually exposed.

- Heavy reading and writing.
- Long, fragile code pipelines.
- Tight reproducibility constraints.
- Long feedback loops with referees and coauthors.

Korinek's claim (Korinek, 2023, 2024)

- LLMs already help across ideation, writing, code, math, data, and review.
- Agentic versions push the frontier from *cognitive assist* to *cognitive automation*.

Brynjolfsson, Li, Raymond (Brynjolfsson et al., 2025)

- Productivity gains are concentrated on **less experienced workers**.
- Gains come from *learned best practices*, not raw automation.

Implication for PhD training

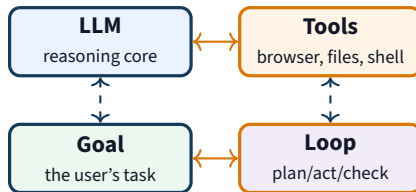
- A young researcher who ignores agents loses a comparative advantage.
- A senior researcher who delegates without controls loses scientific control.

Part 2 – Concepts and vocabulary

What is an agent?

**LLM + tools
+ goal + loop**

An agent is not a smarter chatbot.
It is a workflow with partial autonomy.

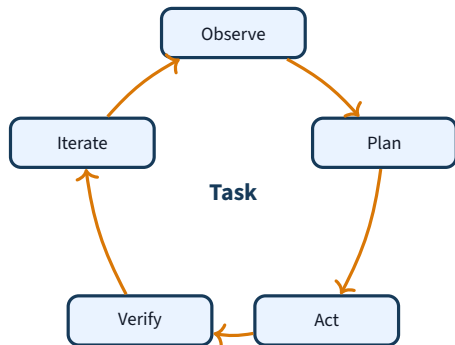


Chatbot vs agent

	Chatbot	Agent
Main action	Responds	Plans and acts
Tools	Optional	Central
Files	Usually attached	Often browsed and edited
Failure mode	Wrong <i>answer</i>	Wrong <i>action</i>
Human role	Reader	Supervisor
Cost driver	Tokens read/written	Tokens <i>plus</i> tool calls and retries

Different failure mode $\Rightarrow$ different control needs.

The agent loop, with a real trace



Task: "Re-run Table 2, fix the missing standard errors."

Observe: read do/02_table2.do, tables/T2.tex.

Plan: reghdfe call lacks cluster(id_firm).

Act: edit line 47, re-run stata-mp -b do/02_table2.do.

Verify: log shows clustered SE; T2.tex regenerated, columns 1-3 unchanged within 1e-4.

Iterate: report diff to user; await approval.

Source: Stylised trace from a real Stata project; commands and file names anonymised.

From prompt to task

Prompt

“Explain how to run this regression.”

One question, one answer. No state. No side effect.

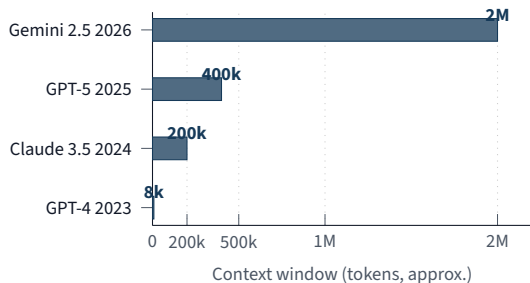
Task

“Inspect the scripts, re-run the model with clustered SE, regenerate the table, compare with the previous version, report what changed.”

A goal, a workflow, a verifiable artefact.

**Agentic AI shifts
the unit of delegation.**

Context, tokens, memory



- **Context** = what the model can see right now (prompt + files + tool outputs).
- **Tokens** are the unit of measurement *and* of cost.
- **Memory** extends context across sessions (project notes, vault, vector store).

More context is not the same as better reasoning.

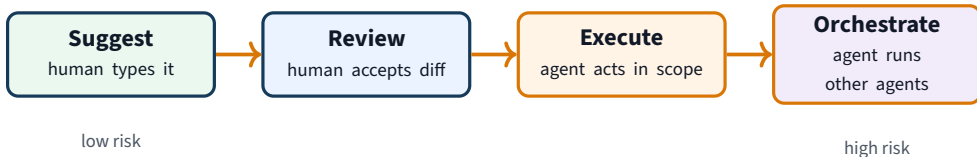
Tools, permissions, sandbox



- **Tool** = something the model can call: browser, shell, files, database, an MCP server.
- **Permission** = the boundary between suggestion and action.
- **Sandbox** = a controlled environment where mistakes are reversible.

Permissions are the practical meaning of control.

The autonomy spectrum



Pick the lowest level that gets the job done.

For research, “review” and “execute” cover almost everything; “orchestrate” is for power users.

Reliability vocabulary you will keep hearing

Hallucination	A confident output that does not correspond to reality (a citation, a number, a function).
Drift	The model's outputs change over time as providers update underlying weights or settings.
Automation bias	The human tendency to trust automated outputs more than they deserve (Skitka et al., 1999).
AI sloppiness	Output that is fluent but lazy: generic phrasing, weak structure, vague claims, missing citations, or polished text that has not really been checked.
Reward hacking	The agent finds a shortcut that satisfies the metric but defeats the goal.
Sycophancy	The model agrees with the user even when the user is wrong.

These are research-process risks, not just AI quirks.

Part 3 – Four interface families

Where you actually meet agentic AI

1. Chat / app

ChatGPT, Claude, Gemini, Grok
talk, think, write, act

2. Research / browser

Perplexity, Elicit, NotebookLM
find, triage, summarise

3. IDE agent

Cursor, Copilot, Antigravity, Codex
code, refactor, test

4. CLI / local

Claude Code, Codex CLI, Ollama
automate, batch, integrate

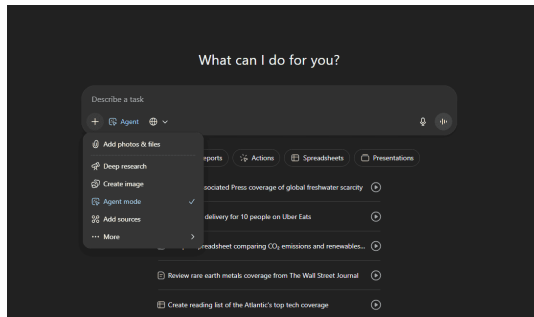
Choose the environment by task, not by brand.

1. Chat / app environments

- ChatGPT, Claude, Gemini, Grok and similar apps.
- Best for thinking, writing, summarising, light analysis.
- Weakness: limited file control, hidden limits, reproducibility.

Agentic versions exist

- “Agent mode”, “Operator”, browser tools, file connectors.
- Same UI, but the model now *acts*, not only *answers*.



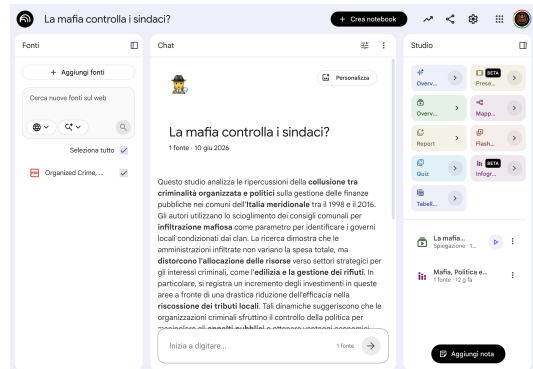
2. Research and browser assistants

Examples

- **Perplexity**: AI search with citations.
- **Elicit**: literature search over OpenAlex / Semantic Scholar.
- **NotebookLM**: ground answers in user-uploaded sources.
- **Deep Research** modes inside major chat apps.

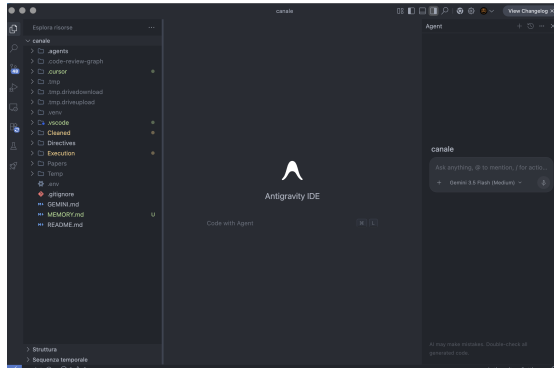
Use

- Source discovery, evidence extraction, paper triage, background memos.



3. IDE agents

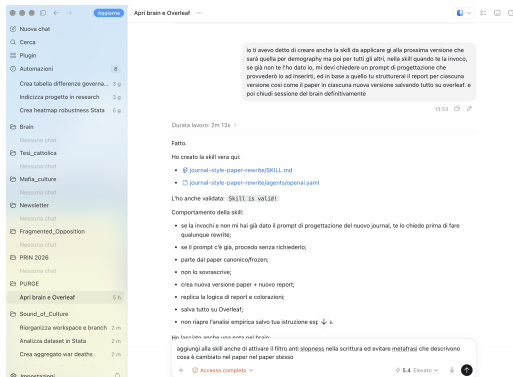
- Cursor, GitHub Copilot Agent, Antigravity, Codex inside competing IDEs
- Best when the object is a codebase.
- Review happens through diffs, tests and file changes.



3.1 Desktop agent apps: the IDE boundary is blurring

Not only chat in a browser

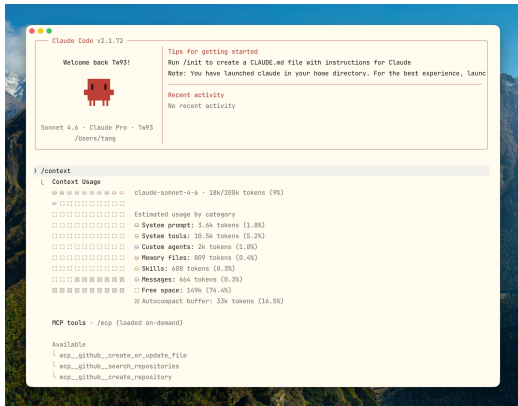
- Cursor desktop app, Google Antigravity, Codex app/extension, Claude Code, and similar tools.
- They can open folders, inspect files, run commands, edit code, and manage tasks.
- The app becomes a control plane around the project, not just a place to ask questions.



Source: Codex desktop app screenshot captured by Marco Le Moglie, 23 June 2026.

For researchers, the relevant question is: *can I review what changed?*

4. CLI agents (and local models)



- Claude Code, OpenAI Codex CLI, Gemini CLI
- They can inspect folders, run commands, edit code, manage processes.
- They consume fewer tokens

Local models

- Ollama, LM Studio, plus open-weight models (Llama, Qwen, Mistral).
- Slower and weaker than frontier, but *private*.

When local is the right answer

Local pays off when

- Data cannot leave the institution.
- Tasks are repetitive and the cost would explode in the cloud.
- You need full reproducibility and version freezing.

Local hurts when

- Tasks need frontier reasoning.
- Hardware is modest (laptop CPU only).
- Multi-step agentic workflows: open models still lag.

For most research, cloud frontier + careful data hygiene wins.

Comparing the main ecosystems, without religion

OpenAI

Generalist chat + Agent + Codex
app/CLI

watch: cost, limits, plan churn

Anthropic

Careful code (Claude Code), strong
defaults

watch: usage caps, restricted regions

Google

Long context + Search + Antigravity +
Gemini CLI

watch: product churn, naming

Cursor

IDE-native, multi-model, agent-first

watch: subscription / usage split

Also: Grok / xAI; Perplexity / Elicit / NotebookLM for search; Llama / Qwen / Mistral for local.

Choose by task, not by brand

Thinking, writing, light analysis

Source discovery, evidence triage

Coding, replication packages, table generation

Batch jobs, scripted pipelines, server work

Sensitive data, full reproducibility

Chat app (any frontier).

Research assistant + chat for synthesis.

IDE agent or desktop agent app.

CLI agent.

Local model where feasible; otherwise enterprise plan with no training.

Most real workflows combine two or three of the solutions above.

Part 4 – Cost, privacy, governance

Subscription vs API

Subscription (consumer / Pro / Team)

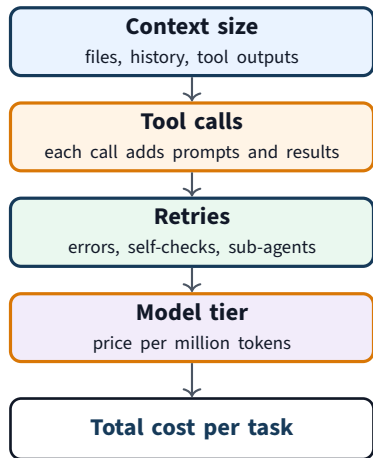
- Flat monthly price.
- Limits are often soft, dynamic, hard to compare.
- “Unlimited” is rarely uncontrolled: throttling, queueing, capability gating.

API / metered usage

- Cost is measurable per token or call.
- Easier to budget, easier to overspend.
- Required for IDE/CLI agents that call many tools.

For agents, “unlimited” rarely means uncontrolled.

The four levers that drive your bill



- **Context** grows with files, browsing, history.
- **Tool calls** multiply with autonomy.
- **Retries** are silent: errors, validation, sub-agents.
- **Model tier** can differ $10\times$ – $50\times$ in price per token.

A chat session ignores three of these. An agent uses all four.

What a real cost spike looks like

Workload	Description	Tokens (approx.)
Chat question	“Explain heteroskedasticity in two lines.”	~ 0.5k
Long reading	“Summarise this 30-page PDF.”	~ 25k
IDE one-shot edit	“Fix this regression call.”	~ 8k
IDE agent task	“Re-run all tables and verify.”	~ 80–200k
Multi-agent research run	“Replicate Section 4 end-to-end.”	~ 500k–2M

Same UI. Two orders of magnitude difference in cost.

Does it train on my data?

The honest answer:

It depends on the *provider, plan, workspace* and *settings*.

- Consumer plans: often **default to training** unless you opt out.
- Business/Enterprise plans: usually **do not train** on your data by default.
- API: usually no training, but check retention policy.

What to verify

- Account type (personal vs business).
- Workspace data-use settings.
- Region and applicable law (GDPR, US sectoral).
- Tool-specific data flow (browsers, retrieval, sub-agents).

Settings change. Verify, do not assume.

Source: See provider documentation: OpenAI (2026); Anthropic (2026); Google (2026); Cursor (2026).

Data risk in economics research

High risk	Microdata, administrative data, embargoed data, confidential referee reports, unpublished IRB-restricted material.
Medium risk	Unpublished drafts, grant ideas, coauthors' working notes, identifiable interview transcripts.
Lower risk	Public documentation, synthetic examples, teaching material, published papers.

When in doubt: anonymise, abstract, or use a sandbox.

Institutional and IRB considerations

Things to clarify with your institution

- Approved vendors and data-processing agreements.
- Whether AI processing of human-subjects data is covered by existing IRB protocols.
- Storage, retention, and right-to-erasure obligations.

Things to clarify in journals

- Disclosure requirements for AI use.
- Reproducibility expectations: AEA Data Editor and similar.
- Authorship: an agent is not an author.

The rules are still moving. Document what you did.

Source: See Vilhuber (2020) on reproducibility norms; consult AEA Data Editor guidance for the latest disclosure standards.

Practical safeguards that actually work

Budget and pace

- Set an explicit monthly budget per project.
- Ask the agent for a plan *before* execution.
- Use a cheaper model for scanning, frontier for the hard step.

Containment

- Sandbox: run agents inside a constrained folder/VM.
- Read-only mode for first passes.
- Kill switch: easy to interrupt; nothing irreversible without confirmation.

Safety is mostly process design, not magic settings.

Part 5 – Reliability and control

Hallucinations are not only fake citations

Citation hallucination	A plausible but nonexistent paper, dataset, or table.
Code hallucination	A function that runs, but implements the wrong logic.
Statistical hallucination	Correct syntax, wrong interpretation (e.g., wrong fixed effects).
Workflow hallucination	Task marked complete without real verification (no test, no log, no diff).

In agents, the most dangerous hallucination is “done”.

Source: Hallucination taxonomy adapted from Ji et al. (2023).

Automation bias and overtrust

- Humans systematically over-trust automated outputs (Skitka et al., 1999).
- The smoother and more confident the output, the stronger the bias.
- AI outputs are smoother and more confident than human drafts.

What this looks like in practice

- Accepting a regression specification “because the output table looks reasonable”.
- Copy-pasting a literature summary without reading the cited papers.
- Skipping unit tests because the agent “already checked”.

Polished prose is not validated reasoning.

Verification by design



Verification cannot be an afterthought; it must be a slot in the workflow.

For research code: at least one test per table, every change as a commit, every command in a log.

When *not* to use an agent

Identification	The hardest, most causal-inference-specific step.
Confidential data	Where the privacy boundary is unclear.
Final claims	The exact wording of theorems, abstracts, conclusions.
Trust-critical artefacts	Replication packages submitted to a data editor.
Disagreements with coauthors	Resolve by talking, not by prompting.

Delegating these does not save time; it transfers risk.

The local alignment problem

**The agent optimises for completing the task.
You care about doing good research.**

- Plausible code is not valid code.
- A complete table is not a correct table.
- A polished paragraph is not a justified claim.

The alignment gap is not in 2030; it is in your next prompt.

Part 6 – Agentic architecture

Rules are prompts that stay.

- Project style and conventions (naming, structure, citation format).
- What the agent must *not* do (don't push to main, don't touch raw/, don't change identification).
- How to verify “done” (tests, logs, diff, peer review).

AGENTS.md | CLAUDE.md | CURSOR.md
GEMINI.md | project instructions

Memory has layers



Each layer answers a different question:

What did we just say? | What is this project? | What did I decide before? | How is everything connected?

In a research workflow, the project layer is the most under-used.

Knowledge base or RAG?

RAG – Retrieval Augmented Generation (Lewis et al., 2020)

- Chunk text → embed → store in a vector DB.
- At query time: retrieve the top- k chunks, inject into the prompt.
- **Pros:** scales to huge corpora; automatic.
- **Cons:** opaque chunking, embedding drift, vendor lock-in, hard to audit *why* a chunk was returned.

Knowledge base – Obsidian-style vault (Obsidian, 2026)

- Curated Markdown files with explicit **bi-directional links**.
- Both human and agent read the *same* files.
- **Pros:** explicit provenance, real graph, local-first, plain text forever, hand-curable, git-versioned.
- **Cons:** requires discipline; does not scale to millions of files.

For research, traceability matters more than automation.

Why a curated KB often beats pure RAG (for research)

- **Provenance:** a claim points to a file path and a backlink, not to a hidden vector ID.
- **Hand-curable:** a wrong fact is fixed by editing one line; no re-embedding pipeline.
- **Local-first:** sensitive notes never leave the laptop.
- **Forever-readable:** Markdown survives any provider change.

The combination that works

- Markdown vault as the spine (decisions, briefs, logs).
- Persistent memory CLI (e.g., MEMANTO) for cross-session facts.
- Session capture (e.g., Hivemind hooks) for what just happened.
- Code/observation graph (e.g., CodeGraph, Cavemem) for “how is it connected”.
- Optional RAG on top of the vault for huge PDFs.

Best of both worlds: a KB you control + RAG as an amplifier.

MCP vs API vs CLI

	What it is	Token cost logic
API	A programmatic endpoint: <i>your code</i> calls a service over HTTP.	Usually lowest tool-loading overhead: your code decides what to send.
CLI	A command-line tool: <i>a human or an agent</i> runs commands.	Medium overhead: command help, logs and outputs enter context when needed.
MCP	A standard way to expose tools, files and data sources to agents (Anthropic, 2024b).	Higher upfront overhead: tool names, schemas and descriptions are loaded so the agent can choose among them.

MCP is the plumbing.
CLI is the operation.
API is the integration.

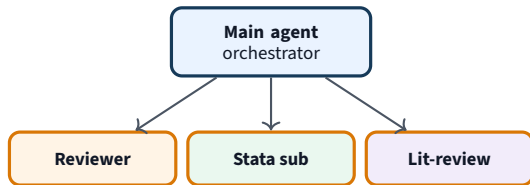
Simple rule: the more tools you expose to the model, the more context you spend before the task even starts.

Sub-agents: specialised workers

- A main agent delegates a sub-task to a smaller, specialised agent.
- Useful for: independent review, parallel exploration, scope isolation.
- Dangerous when sub-agents amplify each other's mistakes (echo chamber).

Concrete examples in research

- “Bugbot” style: review the diff before commit.
- Stata sub-agent: run a regression and report only the table.
- Lit-review sub-agent: pull and triage candidate papers.



Skills, hooks, loops

Skills

Reusable procedures: “how we make a figure”, “how we check tables”, “how we write a referee response”.

- Turn Stata regression output into a paper-ready table.
- Audit a dataset/codebook before the first merge.
- Adapt a frozen draft to a new journal style.

Hooks

Automatic checks before or after an agent acts: lint, format, prose-check, test.

- On session start: load rules, `AGENT_MEMORY.md`, current state.
- After `.tex` edits: prose-check + citation sanity pass.
- Before commit: review diff; block secrets and raw confidential data.

Loops

Recurring tasks: weekly literature scan, monthly replication check, post-revision audit.

- Weekly: scan new papers → triage → vault notes.
- After each new spec: robustness block → appendix refresh.
- Post-referee round: comment → rerun → update tables/figures.

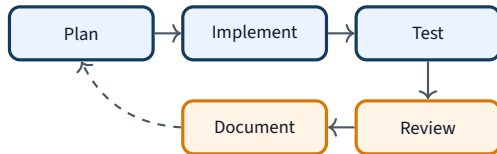
Best use: turn tacit lab habits into explicit, reusable workflows.

Orchestration patterns and the canonical loop

Three common patterns

- **Pipeline** – one agent, sequential phases.
- **Debate** – two agents propose, a third arbitrates.
- **Parallel** – many sub-agents on independent slices.

Canonical agent loop for a research task



Good prompts ask the agent to move through these phases explicitly.

Part 7 – Case studies teaser

What you will (probably) see in detail next session

Five recurring economics workflows where agents help.

- Literature review.
- Data and codebook documentation.
- Stata / R / Python debugging.
- Econometric tables and robustness.
- Referee reports, grants, teaching.

Today: a tour.

- One slide per case, with a real example and a concrete agent task.
- Next session: a full walk-through with my own code, data, and screen.

Same workflow phases (Plan – Implement – Test – Review – Document) across all cases.

Case 1 – Literature review

Agent helps with

- Search-string design and inclusion rules.
- Per-paper summaries and comparison tables.
- Citation checking (does this paper exist? does it say this?).

Human keeps

- What counts as “relevant”.
- How the literature shapes the research design.

Concrete example

Build a comparison table on text-as-data in macro, anchored to Hansen, McMahon, Prat (2018): agent extracts methods, data, identification, sample period for 30 papers from Elicit + Semantic Scholar.

Why it works. The unit of delegation is small, verifiable, and the human still owns the inclusion criteria (Korinek, 2023).

Case 2 – Data and codebook documentation

The agent turns messy data work into explicit documentation.

- Build a codebook from raw variable names and do-files.
- Detect undocumented transformations.
- Write a replication README aligned with AEA data-editor expectations (Vilhuber, 2020).

Concrete example

Stata project with 22 do-files and 7 datasets. Agent runs each do-file, tracks the lineage of every variable across files, and produces:

- `codebook.md` (one section per dataset);
- `lineage.svg` (graph of file → dataset → dataset);
- `README.md` (one-command rebuild).

Pitfall. The agent will document what it sees; if a transformation is in a missing do-file, it disappears silently.

Case 3 – Stata / R / Python debugging

Good agent task

“Find the minimal failing example that reproduces this error and propose a patch.”

What works in practice

- Use an IDE agent on the repo, not a chat app.
- Ask for a *plan* before any edit.
- Demand a regression test added with the fix.

Bad agent task

“Fix my project.”

Stata example via `mcp-stata`

- Agent calls Stata over MCP, runs the broken do-file, captures the log, isolates the offending line, and proposes a patch with a unit-style test on the affected variable.

Narrow the problem before you give autonomy.

Case 4 – Econometric tables and robustness

Where agents really help

- Table-export code and consistent column ordering.
- Specification curves: many models, one figure.
- “Which coefficients moved across robustness checks?”.

Where they must *not* be in charge

- Picking the preferred specification.
- Choosing controls to obtain a desired sign.
- Outsourcing identification.

Inspect logs, sample sizes, and dropped observations every time.

Case 5 – Referee reports, grants, teaching

Referee report	Organise comments, detect internal contradictions, draft tone-neutral phrasing. <i>Never delegate the verdict.</i>
Grant proposal	Work packages, risk tables, timelines, lay summaries; consistency checks across sections.
Teaching	Slides, exercises, rubrics, translations, “minimum viable example” for a method.

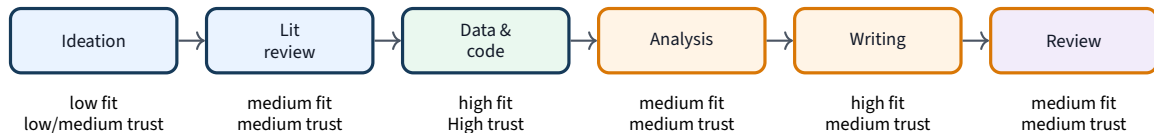
This very deck was assembled with an agentic workflow under explicit human review.

What humans must keep

Theory	What is the economic mechanism?
Identification	What variation identifies the effect?
Measurement	What does the variable really capture?
Interpretation	What can and cannot be claimed?
Responsibility	Who signs the paper?

No agent should ever own these.

Where agents fit across the pipeline



Agents are useful across the pipeline, but not equally trustworthy at every stage.

What we will (probably) do in Session 2

- We will review some of my current projects to see what agents can and cannot do.
- We will learn different lessons from different frictions: writing slides, building datasets, rewriting papers, and creating tables from data.
- We will live-screen selected workflows: initial setup, requests, prompts, control checks, logs, and review.
- If useful, we can also take a quick research idea from the room and test how far an agent can push it in a short session.

Part 8 – Closing

A provocation worth carrying out of the room

If Anyone Builds It, Everyone Dies.

- Yudkowsky and Soares (2025) offer one of the strongest pessimistic formulations of advanced-AI risk.
- Whether or not you accept the conclusion, the right question to keep is: *what happens when autonomy exceeds control?*

**For our daily research, the lesson is modest but firm:
never delegate without a control structure.**

Five take-home messages

-
- 1 Agents are workflows with partial autonomy, not magic assistants.
 - 2 The right interface depends on the task, not the brand.
 - 3 Privacy, cost and reproducibility are design choices, not afterthoughts.
 - 4 Memory is layered: a curated Markdown KB plus a few targeted memory tools beats blind RAG.
 - 5 The best human role is supervisor, theorist, and final judge.
-

Thank you.

References I

Anthropic. The Anthropic Economic Index. <https://www.anthropic.com/economic-index>, 2024a. Accessed 2026-06-23.

Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2024b. Accessed 2026-06-23.

Anthropic. Claude pricing and trust center. <https://www.anthropic.com/pricing>, 2026. Accessed 2026-06-23.

Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond. Generative AI at work. *Quarterly Journal of Economics*, 2025. doi: 10.1093/qje/qjae044.

Cursor. Cursor pricing and product overview. <https://cursor.com/pricing>, 2026. Accessed 2026-06-23.

Google. Google AI plans. <https://one.google.com/about/google-ai-plans>, 2026. Accessed 2026-06-23.

Stephen Hansen, Michael McMahon, and Andrea Prat. Transparency and deliberation within the FOMC: A computational linguistics approach. *Quarterly Journal of Economics*, 133(2):801–870, 2018. doi: 10.1093/qje/qjx045.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. doi: 10.1145/3571730.

Anton Korinek. Language models and cognitive automation for economic research. *Journal of Economic Literature*, 61(4):1281–1317, 2023. doi: 10.1257/jel.20231736.

Anton Korinek. LLMs learn to collaborate and reason: Implications for economists. Working paper, Brookings Institution, 2024. URL <https://www.brookings.edu/articles/llms-learn-to-collaborate-and-reason>.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* 33, 2020.

Obsidian. Obsidian help and documentation. <https://help.obsidian.md>, 2026. Accessed 2026-06-23.

OpenAI. Chatgpt pricing and business data controls. <https://openai.com/business/chatgpt-pricing>, 2026. Accessed 2026-06-23.

Linda J. Skitka, Kathleen L. Mosier, and Mark Burdick. Does automation bias decision-making? *International Journal of Human-Computer Studies*, 51(5):991–1006, 1999. doi: 10.1006/ijhc.1999.0252.

Lars Vilhuber. Reproducibility and replicability in economics. *Harvard Data Science Review*, 2(4), 2020. doi: 10.1162/99608f92.4f6b9e67.

Eliezer Yudkowsky and Nate Soares. *If Anyone Builds It, Everyone Dies*. Little, Brown, 2025.